



Documento Técnico, Entrega Obligatoria N.º 1

Materia	Desarrollo de Aplicaciones II, 3.4.218, Comisión Lunes TM
Dominio	MediConecta, turnos médicos y telemedicina
Alcance de esta entrega	ServicioDeUsuarios · ServicioDeTurnos · ServicioDeHistoriaClinica
Stack	Jakarta EE 10 (JSF/PrimeFaces, EJB, JPA/Hibernate) sobre WildFly + PostgreSQL (Supabase)
Fecha	14/09/2026

1. Introducción y alcance

MediConecta es una plataforma que conecta pacientes con profesionales de salud independientes y pequeñas clínicas para la gestión de turnos presenciales y telemedicina. El sistema completo, descrito en la consigna del TP Integrador, contempla ocho componentes de negocio e integraciones externas síncronas (SOAP/REST) y asincrónicas (mensajería).

Se implementaron y desplegaron **tres componentes completos**: `ServicioDeUsuarios`, `ServicioDeTurnos` y `ServicioDeHistoriaClinica`. Deliberadamente quedan fuera de esta entrega las integraciones externas (obra social vía SOAP, pasarela de pago y videoconsulta vía REST) y la mensajería asincrónica (notificaciones), que corresponden a la Unidad IV en adelante y se incorporarán de forma incremental en la Entrega Parcial N.º 2 (19/10) y la Entrega Obligatoria N.º 2 (09/11), tal como exige el Anexo del TP.

1.1 Checklist de la Unidad I–II–III cubierto en esta entrega

- ✓ Al menos 3 componentes completamente implementados y desplegados, cada uno con arquitectura en capas.
- ✓ Al menos 1 componente *stateful* (`ServicioDeTurnos`) y 2 *stateless* (`ServicioDeUsuarios`, `ServicioDeHistoriaClinica`), con evidencia de que el contenedor gestiona su ciclo de vida.
- ✓ Tres patrones de diseño distintos, aplicados y justificados: DAO, Facade y Factory.
- ✓ Seguridad declarativa mínima: autenticación y autorización por rol en una operación sensible (carga de diagnósticos/recetas en la historia clínica).

2. Arquitectura general

El sistema sigue una arquitectura en tres capas, presentación, negocio y datos, replicada en cada componente, sobre un servidor de aplicaciones Jakarta EE (WildFly). La capa de presentación no toma ninguna decisión de negocio: solo valida formato de entrada y delega en la capa de negocio a través de una interfaz local de EJB. La capa de negocio aplica las reglas del dominio, orquesta procesos y gobierna transacciones (`@TransactionAttribute`) y seguridad (`@RolesAllowed`). La capa de datos aísla el acceso a PostgreSQL detrás del patrón DAO.

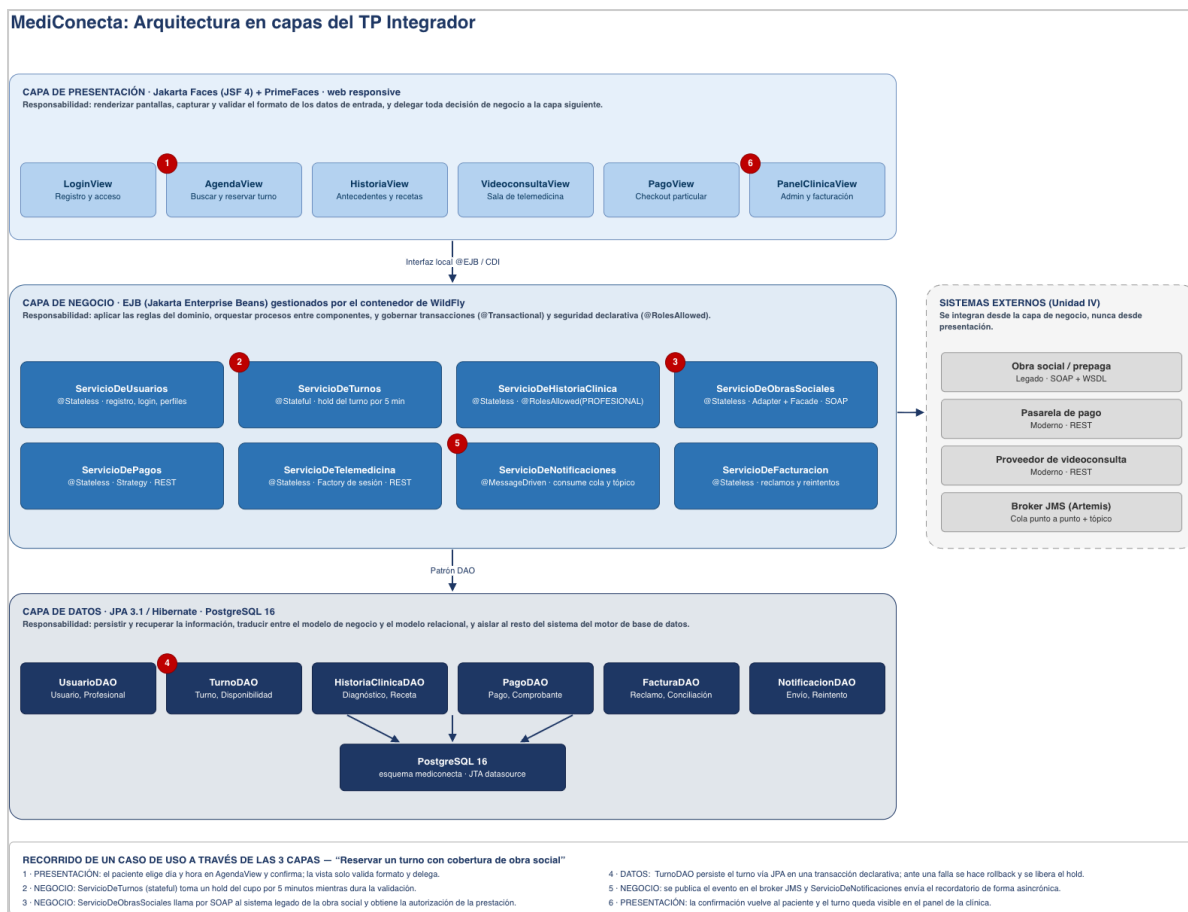


Fig. 1, arquitectura en capas planificada para el sistema completo. Los componentes con borde numerado 2 y 3 (ServicioDeUsuarios, ServicioDeTurnos, ServicioDeHistoriaClinica) son los implementados en esta entrega; el resto se incorpora en entregas posteriores.

2.1 Por qué estos tres componentes primero

Se eligieron porque forman, por sí solos, un caso de uso de punta a punta completamente funcional sin depender de ningún sistema externo: un paciente se registra, reserva un turno con un profesional, y el profesional documenta la atención en la historia clínica. Esto evita mostrar componentes aislados sin integración real entre sí, que es justamente lo que la consigna penaliza ("un sistema técnicamente simple pero mal justificado va a rendir peor que uno más acotado con decisiones sólidas").

3. Componentes implementados

3.1 ServicioDeUsuarios

Atributo	Detalle
Responsabilidad	Registro y autenticación de pacientes y profesionales; hashing de contraseñas; alta de perfiles.
Tipo de EJB	@Stateless
Interfaz local	ServicioDeUsuarios (@Local)
Operaciones	registrar(Rol, DatosRegistro, password) , buscarPorEmail(email) , listarProfesionales()
Capa de datos	UsuarioDAO (CDI @ApplicationScoped , EntityManager inyectado)
Entidades	Usuario (abstracta) → Paciente , Profesional (herencia SINGLE_TABLE)

3.2 ServicioDeTurnos

Atributo	Detalle
Responsabilidad	Publicación de disponibilidad, búsqueda, reserva temporal ("hold") y confirmación de turnos.
Tipo de EJB	@Stateful con @StatefulTimeout(5, MINUTES)
Interfaz local	ServicioDeTurnos (@Local)
Operaciones	publicarDisponibilidad , buscarDisponibilidad , reservarHold , confirmar , getTurnoEnHold , buscarConfirmadosDePaciente/DeProfesional
Capa de datos	TurnoDAO
Entidad	Turno (estados: DISPONIBLE, HOLD, CONFIRMADO, CANCELADO; con @Version para control optimista)

3.3 ServicioDeHistoriaClinica

Atributo	Detalle
Responsabilidad	Consulta y alta de antecedentes, diagnósticos y recetas. Datos de alta sensibilidad: exige seguridad declarativa.
Tipo de EJB	@Stateless
Interfaz local	ServicioDeHistoriaClinica (@Local)
Operaciones	obtenerDePaciente(pacienteId) , agregarEntrada(pacienteId, profesionalId, turnoId, tipo, descripcion)
Capa de datos	HistoriaClinicaDAO
Entidades	HistoriaClinica (1 a 1 con Paciente) → EntradaClinica (DIAGNOSTICO / RECETA)
Seguridad	@RolesAllowed("PROFESIONAL") en agregarEntrada ; @RolesAllowed({"PACIENTE", "PROFESIONAL"}) en la lectura

4. Componente stateful vs. stateless: justificación

ServicioDeTurnos es el único componente stateful porque la reserva de un turno necesita conversación con estado: cuando el paciente selecciona un horario, el sistema debe "reservar" ese cupo por un tiempo acotado (5 minutos) mientras el paciente decide confirmar, sin que otro paciente pueda tomarlo en simultáneo. Esa ventana de decisión es, por definición, estado conversacional atado a un cliente particular, el caso de libro para un EJB @Stateful .

Evidencia de ciclo de vida gestionado por el contenedor

ServicioDeTurnosBean usa @PostConstruct para inicializar la conversación, @PrePassivate / @PostActivate para registrar pasivación (el contenedor puede serializar la instancia a disco si el servidor tiene muchas conversaciones abiertas), y sobre todo @PreDestroy combinado con @StatefulTimeout(value = 5, unit = TimeUnit.MINUTES) . Si el paciente abandona la reserva sin confirmar, el **contenedor EJB**, no una tarea programada nuestra, destruye la instancia al vencer el timeout, y el callback @PreDestroy libera automáticamente el turno (vuelve a estado DISPONIBLE). Este comportamiento no se puede lograr con un bean stateless ni con un simple objeto en sesión HTTP, depende del Timer Service del EJB container.

ServicioDeUsuarios y ServicioDeHistoriaClinica son stateless porque cada operación que exponen es atómica y autocontenida: registrar un usuario, autenticar, consultar o agregar una entrada de historia clínica no requieren recordar nada entre una invocación y la siguiente. Forzarlos a stateful solo agregaría el costo de mantener una instancia por cliente sin ningún beneficio, ya que el contenedor puede pooler instancias stateless libremente entre requests.

5. Patrones de diseño aplicados

5.1 DAO (Data Access Object)

Problema que resuelve: desacoplar la lógica de negocio (los EJB) del acceso a datos vía JPA/Hibernate, para que un cambio en cómo se persiste una entidad no obligue a tocar la lógica de negocio, y viceversa. **Dónde se aplica:** `UsuarioDAO`, `TurnoDAO` y `HistoriaClinicaDAO`, cada uno encapsula el `EntityManager` y expone métodos de dominio (`buscarPorEmail`, `buscarDisponibles`, `agregarEntrada`) en vez de exponer JPQL o el `EntityManager` directamente a los EJB de negocio. **Alternativa descartada:** inyectar el `EntityManager` directamente en los EJB de negocio; se descartó porque mezclaría consultas JPQL con reglas de negocio y dificultaría testear la lógica de negocio de forma aislada.

5.2 Facade

Problema que resuelve: simplificar la interacción con un subsistema complejo. La reserva de un turno no es una operación atómica sino un pequeño proceso (buscar disponibilidad → tomar hold → validar que nadie más lo tomó → confirmar o expirar) con reglas de concurrencia y estado. **Dónde se aplica:** `ServicioDeTurnosBean` actúa como fachada de ese subproceso: la capa de presentación solo ve cuatro métodos simples (`buscarDisponibilidad`, `reservarHold`, `confirmar`, `getTurnoEnHold`) y nunca necesita conocer que por detrás hay una máquina de estados sobre la entidad `Turno`, un timer de expiración y validaciones de concurrencia. **Alternativa descartada:** exponer directamente el DAO y dejar que la vista orqueste los pasos; se descartó porque dispersaría la lógica de negocio (y su seguridad transaccional) en la capa de presentación.

5.3 Factory

Problema que resuelve: la creación de un `Usuario` varía según el tipo de registro elegido (paciente u profesional), cada uno con campos propios (obra social vs. matrícula y especialidad) y sin que el código cliente necesite conocer las subclases concretas. **Dónde se aplica:** `UsuarioFactory.crear(Rol, DatosRegistro, passwordHasheado)` decide, según el `Rol`, si construye un `Paciente` o un `Profesional`, encapsulando esa decisión fuera de `ServicioDeUsuariosBean`. **Alternativa descartada:** un `if/else` directamente en el EJB de negocio; se descartó porque cada vez que se agregue un nuevo tipo de usuario (por ejemplo, Administrador de clínica en una entrega futura) habría que modificar el EJB en vez de solo la fábrica.

6. Seguridad declarativa

La autenticación se resuelve con `@DatabaseIdentityStoreDefinition` (Jakarta Security), declarado en `ConfiguracionSeguridad`: valida contra la tabla `usuario` (`callerQuery` / `groupsQuery`) y compara contraseñas con `Pbkdf2PasswordHash` (hash + sal + 2048 iteraciones, nunca texto plano). El rol obtenido (`PACIENTE` / `PROFESIONAL`) se propaga al contexto de seguridad de Jakarta EE, y la **autorización** se aplica de forma puramente declarativa con `@RolesAllowed("PROFESIONAL")` sobre `ServicioDeHistoriaClinicaBean.agregarEntrada`, la operación sensible que la consigna exige

proteger: solo un profesional autenticado puede escribir diagnósticos o recetas en la historia clínica de un paciente.

7. Flujo de caso de uso de punta a punta

El siguiente flujo fue probado en vivo, desplegado en WildFly contra una base PostgreSQL real (Supabase), y es el que se muestra en la demo de la defensa oral:

- 1 **Registro y login.** El profesional se registra (`ServicioDeUsuarios.registrar` con `Ro1.PROFESIONAL` , usando `UsuarioFactory`) y luego el paciente hace lo mismo con `Ro1.PACIENTE` . Cada uno inicia sesión con su email y contraseña.
- 2 **Publicación de disponibilidad.** El profesional, autenticado, publica un horario (`ServicioDeTurnos.publicarDisponibilidad`); el turno queda en estado DISPONIBLE.
- 3 **Búsqueda y hold.** El paciente busca la disponibilidad de ese profesional y reserva un turno; el bean stateful abre la conversación y marca el turno en HOLD.
- 4 **Confirmación.** El paciente confirma dentro de la ventana de 5 minutos; el turno pasa a CONFIRMADO. Si no confirma, el `@StatefulTimeout` libera el turno automáticamente.
- 5 **Historia clínica.** El profesional, con el turno ya confirmado, carga un diagnóstico (`ServicioDeHistoriaClinica.agregarEntrada` , protegido por `@RolesAllowed("PROFESIONAL")`). El paciente puede ver su historia en solo lectura.

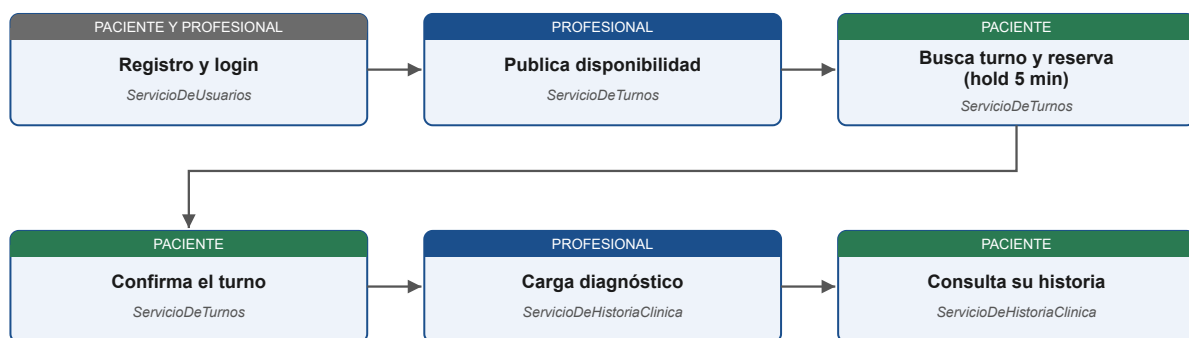


Fig. 2, flujo de negocio completo probado de punta a punta, con el actor y el componente responsable de cada paso.



Fig. 3, login y registro.

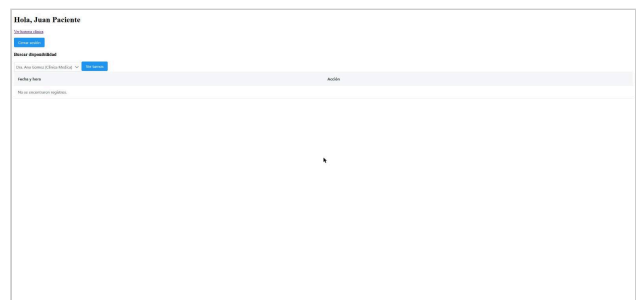


Fig. 4, agenda del paciente autenticado.

Historia clínica			
Fecha	Tipo	Descripción	Profesional
14/09/2026 15:35	DIAGNOSTICO	Control de rutina, sin hallazgos relevantes.	Dra. Ana Gomez

Fig. 5, historia clínica del paciente con una entrada de diagnóstico cargada por el profesional, persistida en PostgreSQL (Supabase).

8. Stack tecnológico y decisiones de diseño

Decisión	Justificación
Jakarta EE 10 sobre WildFly	Requerido por la cátedra como una de las opciones de stack; da acceso nativo a EJB (stateful/stateless con ciclo de vida gestionado), JPA y Jakarta Security sin librerías de terceros para esas piezas.
JSF 4 + PrimeFaces 14	Presentación basada en componentes con binding directo a managed beans CDI, minimizando código de "plumbing" HTTP y dejando la capa de presentación realmente delgada.
PostgreSQL gestionado en Supabase	Evita depender de una instancia de Postgres local para el desarrollo/demo colaborativo del equipo; el driver JDBC estándar de PostgreSQL funciona sin cambios contra Supabase. Se usó el <i>Session pooler</i> de Supabase (compatible IPv4) en lugar de la conexión directa (IPv6 por defecto).
Herencia <code>SINGLE_TABLE</code> para Usuario	Paciente y Profesional comparten identidad, autenticación y rol; una tabla única simplifica el <code>callerQuery</code> / <code>groupsQuery</code> de seguridad declarativa (una sola consulta por email, sin necesidad de JOIN entre tablas de subtipo).
<code>@Version</code> en Turno	Bloqueo optimista para evitar que dos pacientes tomen el mismo turno en una condición de carrera, complementario al hold temporal.

9. Uso de inteligencia artificial generativa

Conforme a lo exigido en la Sección 9 del TP Integrador, se declara el uso de IA generativa (Claude, Anthropic) como asistente de productividad durante esta entrega: generación de boilerplate (entidades JPA, DAOs, vistas JSF), redacción de este documento técnico, y asistencia en el diagnóstico de errores de integración entre Jakarta Security y WildFly 41 (el problema descrito en la Sección 6). Cada integrante del equipo puede explicar y defender individualmente cualquier parte del código o del diseño en la instancia oral, incluidas las decisiones tomadas con asistencia de IA.